

DEVIXO SOLUTIONS · WEB DEVELOPMENT INTERNSHIP

Online Course Platform

LearnHub — Internship Task Report

Student Name: Ghulam Mohiyu Ud Din

Internship Domain: Web Development

Task Number: Task 4

Organization: Devixo Solutions

Email: ghulammohiyudin11@gmail.com

GitHub: github.com/GMD369

LinkedIn: linkedin.com/in/ghulam-mohiyu-ud-din

Date: August 2026

1. Objective

The objective of this task was to design and develop a complete, industry-oriented Online Course Platform (LearnHub) with authentication, role-based access control, and database integration, using the MERN stack (MongoDB, Express, React, and Node.js). The application was required to implement user registration, login, and logout backed by secure JWT sessions; a course module supporting full CRUD for instructors, including ordered lessons with video links and thumbnails; a public course catalog with search, filtering, sorting, and pagination; a simulated checkout flow allowing students to enroll in free and paid courses; and role-specific dashboards for students, instructors, and admins with live statistics. As bonus requirements, the system also needed to persist all course, enrollment, and user data permanently in MongoDB, enforce role-based authorization independently on the client and the server, and present a professional, Udemy-inspired, fully responsive interface.

2. Problem Statement

Independent instructors and small training organizations often lack an affordable, centralized platform to publish courses, manage lessons, and track enrollment — relying instead on ad-hoc file sharing or spreadsheets that offer no access control, no search, and no visibility into who has enrolled in what. There was a need for a web application where instructors can safely create, edit, publish, and remove their own courses and lessons without being able to touch anyone else's; where students can browse, search, and filter a public catalog and enroll (with a simulated payment step for paid courses) without being able to modify course content; and where admins retain full oversight of every user and course on the platform — all through a clean, responsive interface that works as well on a phone as it does on a desktop, and that scales to dozens of courses without becoming difficult to browse.

3. Technologies Used

- React 19 (Vite) for the single-page frontend
- React Router for client-side routing and role-gated route protection
- Tailwind CSS v4 for a themeable, responsive, Udemy-inspired design system
- Axios for API communication with cookie-based credentials
- React Context API for global authentication state
- lucide-react for iconography
- Node.js and Express for the REST API backend

- MongoDB with Mongoose for schema modelling and persistence (hosted on MongoDB Atlas)
- JSON Web Tokens (JWT) stored in an httpOnly cookie for stateless authentication
- bcryptjs for password hashing
- express-validator for server-side request validation
- multer for course thumbnail image uploads
- helmet, express-rate-limit, and express-mongo-sanitize for API hardening
- Git and GitHub for version control; Render (API) and Vercel (frontend) for deployment

4. Implementation Steps

1. **Planning the architecture** — decided on a decoupled frontend/backend structure, three user roles (student, instructor, admin), and JWT-in-httpOnly-cookie authentication shared across both layers.
2. **Building the data models** — designed User, Course (with an embedded, ordered Lesson sub-schema), and Enrollment schemas with Mongoose, including a pre-save hook to hash passwords and a text index on course title/description for search.
3. **Building the authentication API** — implemented register, login, logout, and a session-restore /auth/me endpoint, all validated with express-validator and routed through a centralized error-handling middleware.
4. **Building the course & enrollment API** — implemented paginated, searchable, and filterable course listing, single-course retrieval with enrollment status, full course and lesson CRUD gated by protect + restrictTo('instructor', 'admin') plus an owner-or-admin ownership check, thumbnail upload via multer, and a mock-checkout enrollment endpoint.
5. **Building the React shell** — created an AuthContext that restores the session on page load, a ProtectedRoute component supporting per-route role restrictions, and separate dashboard layouts for student, instructor, and admin.
6. **Building the UI features** — implemented the course catalog with search/category/level/sort filters and pagination, a course detail page with an embeddable YouTube lesson player, the EnrollModal simulated checkout, and instructor tools for creating and editing courses and lessons.
7. **Designing the visual theme** — rebuilt the interface as a Udemy-inspired design system (Tailwind CSS v4 with a custom purple brand palette), including a search-first navbar, course cards with ratings and bestseller badges, a dark hero and course-content section, and a dark multi-column footer, fully responsive across breakpoints.

8. **Seeding and testing data** — wrote seed scripts to populate the database with an instructor account and 30 courses spanning nine categories, each with a real, topic-relevant thumbnail image and a matching YouTube lesson video, so pagination, filtering, and dashboards could be verified against a realistic dataset.
9. **Deployment** — deployed the Express API to Render and the React build to Vercel, and resolved the resulting monorepo build-root, cross-origin cookie, and CORS configuration required for the two services to communicate securely in production.

5. Code Screenshots with Explanations

5.1 JWT Auth & Role Middleware (server/src/middleware/auth.js)

```
export const protect = catchAsync(async (req, res, next) => {
  let token;
  if (req.cookies?.token) {
    token = req.cookies.token;
  } else if (req.headers.authorization?.startsWith('Bearer ')) {
    token = req.headers.authorization.split(' ')[1];
  }
  if (!token) return next(new AppError('You are not logged in.', 401));

  const decoded = verifyToken(token);
  const user = await User.findById(decoded.id);
  if (!user || !user.isActive) {
    return next(new AppError('The user no longer exists.', 401));
  }
  req.user = user;
  next();
});

export function restrictTo(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return next(new AppError('You do not have permission.', 403));
    }
    next();
  };
}
```

This middleware reads the JWT from the httpOnly cookie (or an Authorization header, for non-browser clients) set at login, verifies it, and attaches the authenticated user to the request. `restrictTo` is layered on top of it on every course-mutating route so that only instructor and admin accounts can create or edit courses, while students can still read them — enforcing the same rule the client UI already hides behind role checks, but on the server where it cannot be bypassed.

5.2 Paginated, Searchable & Filterable Course Listing (server/src/controllers/courseController.js)

```
export const getCourses = catchAsync(async (req, res) => {
  const { search, category, level, sort, page = 1, limit = 9, mine } = req.query;
  const query = {};

  if (req.user?.role === 'instructor' && mine === 'true') {
    query.instructor = req.user._id;
  } else {
    query.published = true;
  }
}
```

```

if (search) query.$text = { $search: search };
if (category) query.category = category;
if (level) query.level = level;

const sortMap = { newest: '-createdAt', popular: '-enrollmentCount',
  'price-asc': 'price', 'price-desc': '-price' };
const pageNum = Math.max(1, Number(page));
const limitNum = Math.min(50, Math.max(1, Number(limit)));

const [courses, total] = await Promise.all([
  Course.find(query).populate('instructor', 'name email avatar')
    .sort(sortMap[sort] || '-createdAt')
    .skip((pageNum - 1) * limitNum).limit(limitNum),
  Course.countDocuments(query),
]);

res.json({ success: true, data: { courses,
  pagination: { total, page: pageNum, limit: limitNum, pages: Math.ceil(total / limitNum)
    || 1 } } });
});

```

The listing endpoint combines full-text search, category/level filtering, sorting, and pagination in a single query: a MongoDB text index on the course title and description powers `$text` search, and `skip/limit` compute the correct page slice. The response returns the total record and page counts alongside the page of data, which the React Courses page uses to render its filter bar and pagination controls.

5.3 Auth Session Restore (client/src/context/AuthContext.jsx)

```

const fetchMe = useCallback(async () => {
  try {
    const { data } = await api.get('/auth/me');
    setUser(data.data.user);
  } catch {
    setUser(null);
  } finally {
    setLoading(false);
  }
}, []);

useEffect(() => {
  fetchMe();
}, [fetchMe]);

```

On first load, the React app calls the `/auth/me` endpoint with credentials attached; the `httpOnly` cookie set during login is sent automatically, so a page refresh keeps the user signed in without storing any token in client-side JavaScript, avoiding exposure to XSS-based token theft. `ProtectedRoute` reads this context and redirects to `/login` whenever user is `null` once loading completes.

5.4 Role-Gated Route Protection (client/src/components/ProtectedRoute.jsx)

```
export default function ProtectedRoute({ roles }) {
  const { user, loading } = useAuth();
  const location = useLocation();

  if (loading) return <Spinner className="h-10 w-10" />;
  if (!user) {
    return <Navigate to="/login" replace state={{ from: location }} />;
  }
  if (roles && !roles.includes(user.role)) {
    return <Navigate to="/dashboard" replace />;
  }
  return <Outlet />;
}
```

A single reusable route guard covers both authentication and authorization: routes nested under it in the router (e.g. `/instructor/*`, `/admin/*`) render only when a user is signed in and, if an allowed `roles` list is supplied, only when that user's role is in it — otherwise they are redirected to their own dashboard. This is a UX convenience, not a security boundary by itself: the same rule is independently enforced server-side by `restrictTo`, so a student account cannot perform a mutation even by calling the API directly.

6. Output / Result Screenshots

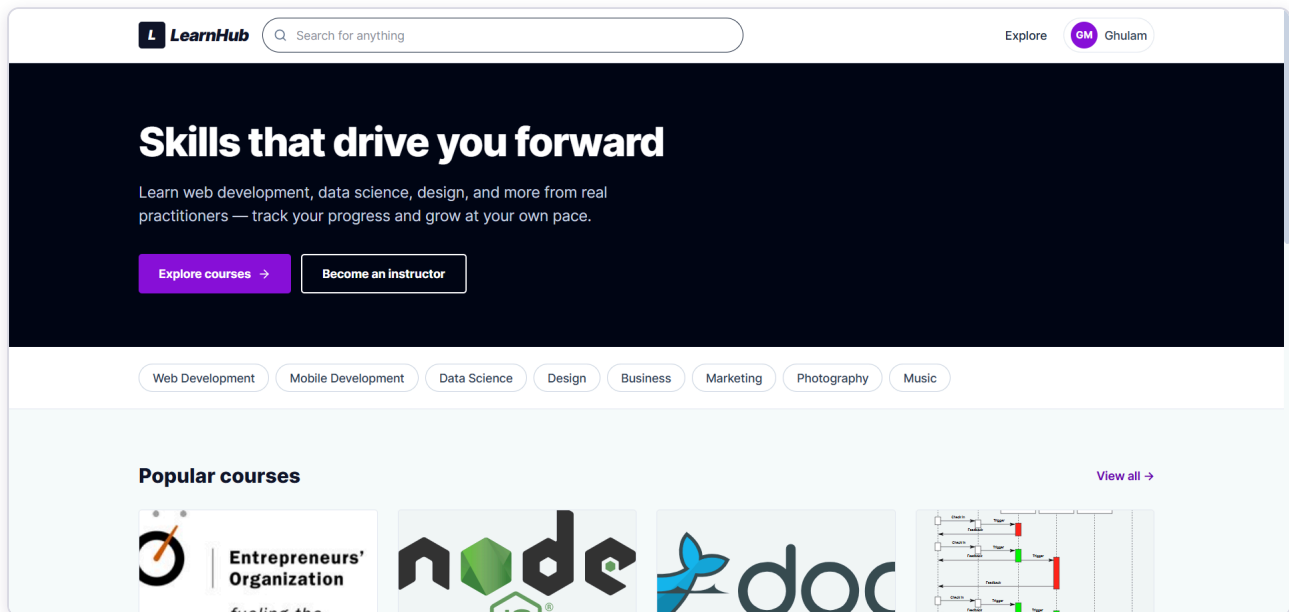


Figure 6.1 — Homepage hero, showing the dark Udemy-inspired banner, headline, call-to-action buttons, and category pill navigation.

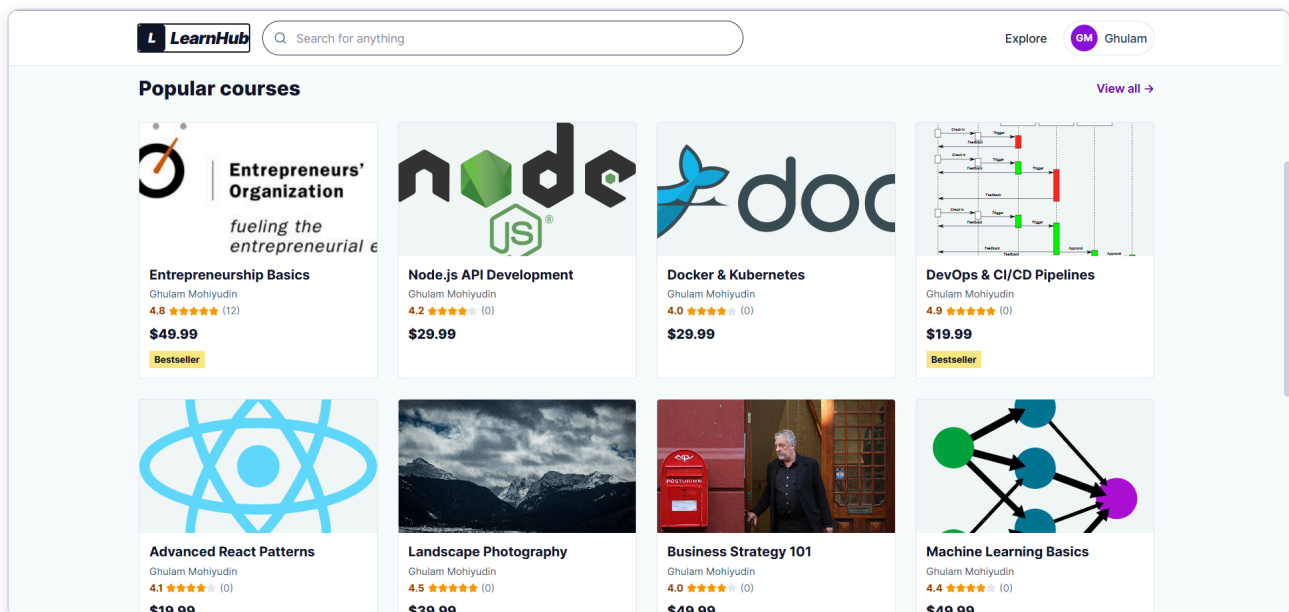
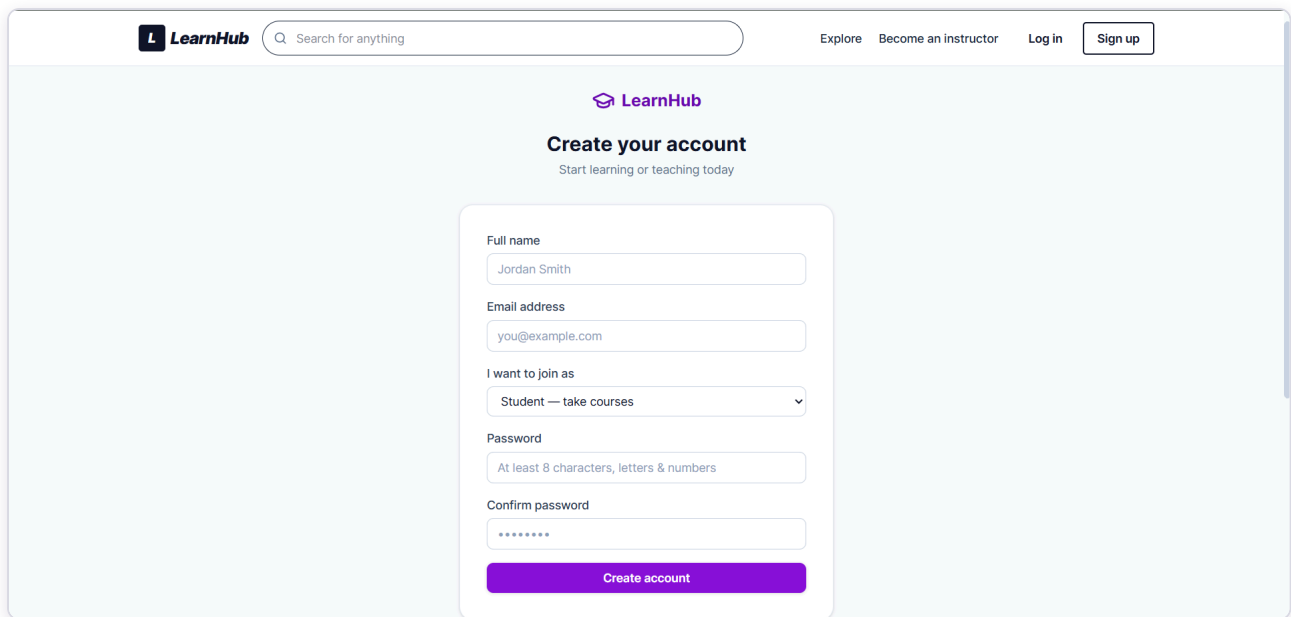
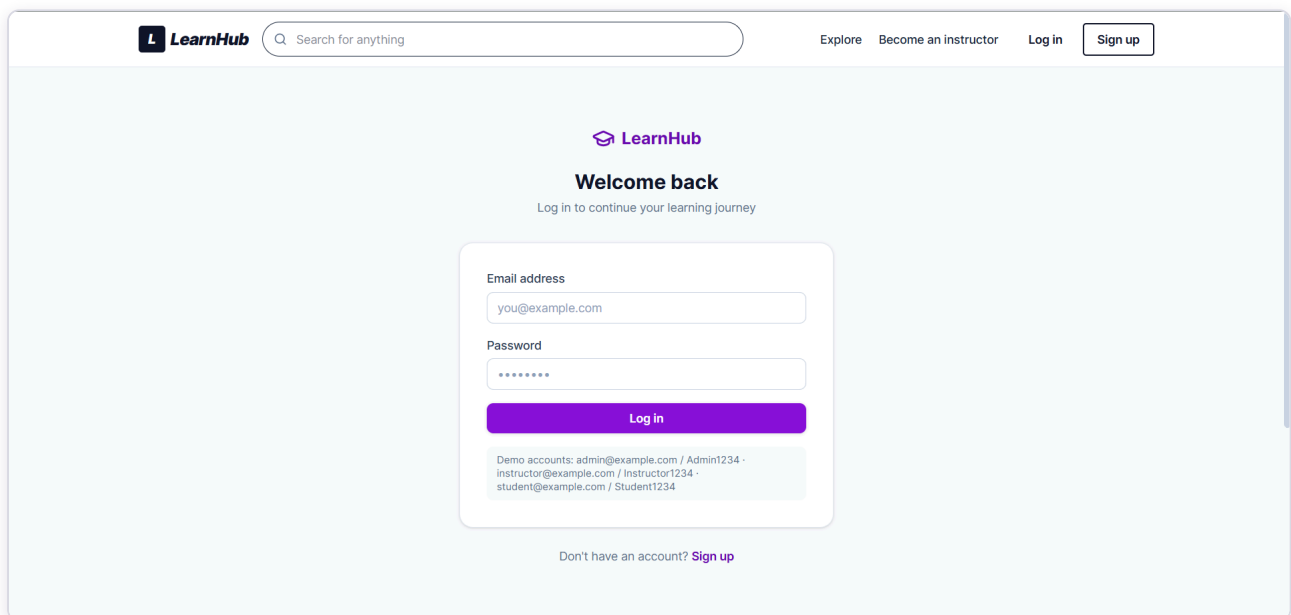


Figure 6.2 — Homepage popular courses section, showing the course card grid with thumbnails, ratings, and pricing pulled from the seeded catalog.



The registration page features a top navigation bar with the LearnHub logo, a search bar, and links for Explore, Become an instructor, Log in, and Sign up. The main heading is "Create your account" with the subtext "Start learning or teaching today". The registration form includes fields for Full name (Jordan Smith), Email address (you@example.com), a role selection dropdown (Student — take courses), Password (with a hint: At least 8 characters, letters & numbers), and Confirm password (masked with dots). A purple "Create account" button is at the bottom of the form.

Figure 6.3 — Registration page, showing the account creation form with name, email, password, and role selection.



The login page features the same top navigation bar as the registration page. The main heading is "Welcome back" with the subtext "Log in to continue your learning journey". The login form includes fields for Email address (you@example.com) and Password (masked with dots), followed by a purple "Log in" button. Below the button, a box lists demo accounts: admin@example.com / Admin1234, instructor@example.com / Instructor1234, and student@example.com / Student1234. At the bottom, a link says "Don't have an account? Sign up".

Figure 6.4 — Login page, showing the branded auth card with email and password fields and a link to the registration page.

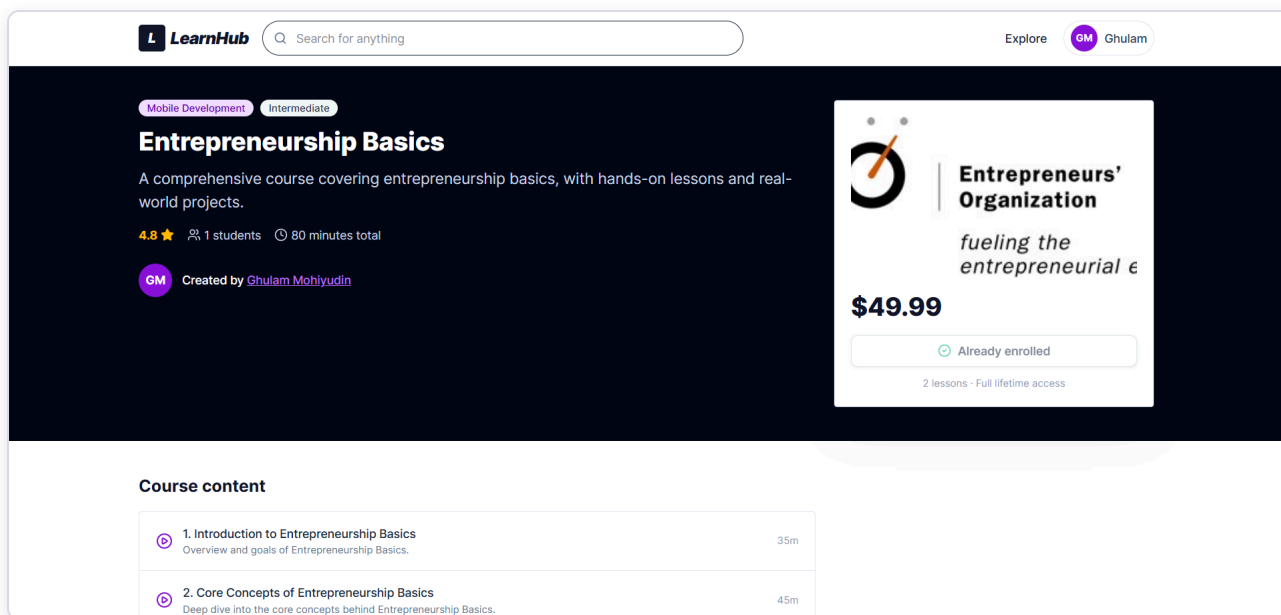


Figure 6.5 — Course detail page, showing the dark hero with rating, student count and instructor, the floating purchase card, and the expandable lesson list with an embedded YouTube player.

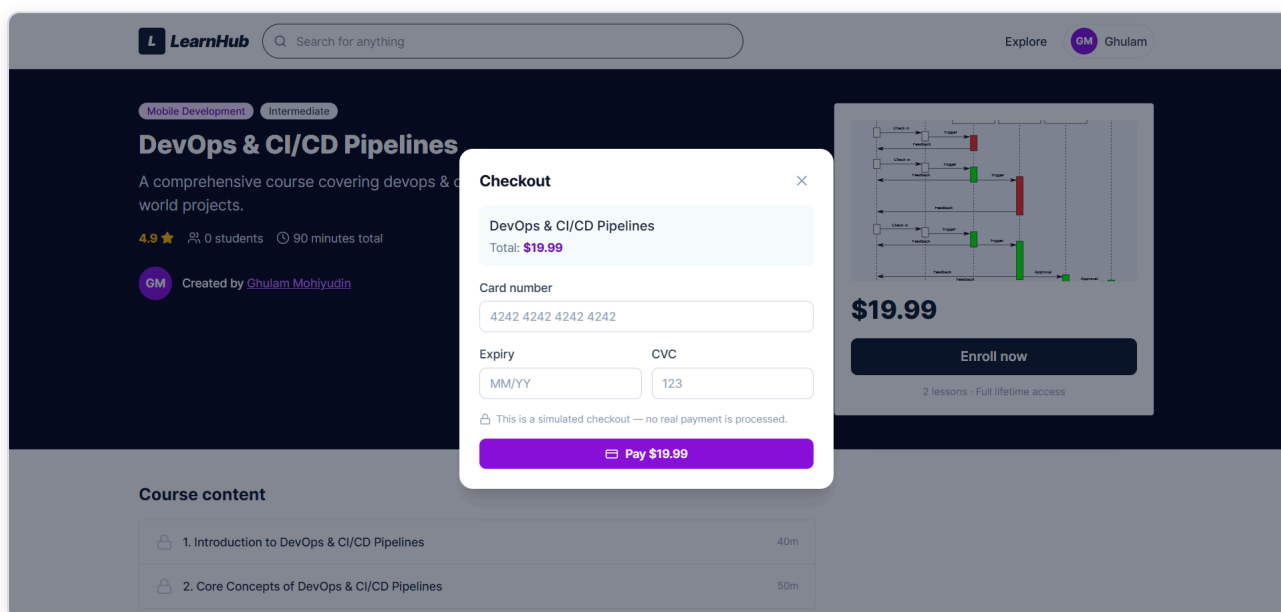


Figure 6.6 — Simulated checkout modal (EnrollModal), showing the mock card-payment form used to enroll in a paid course without a real payment processor.

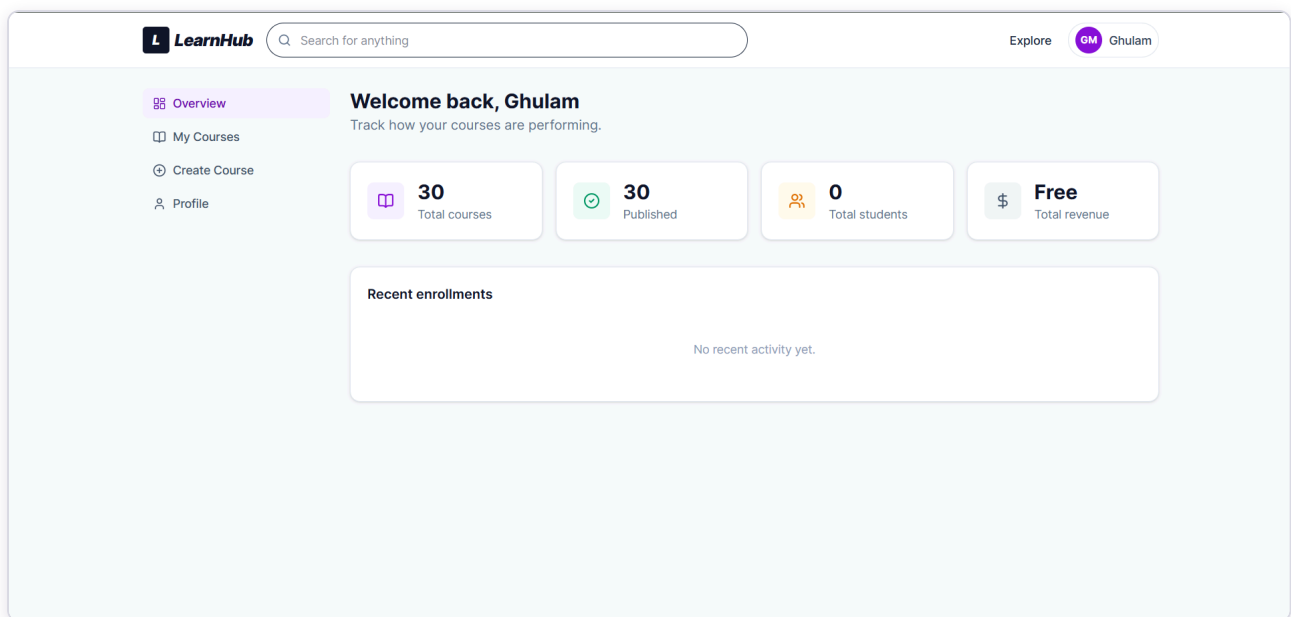


Figure 6.7 — Instructor dashboard overview, showing course and enrollment statistics and a recent-activity panel for the signed-in instructor.

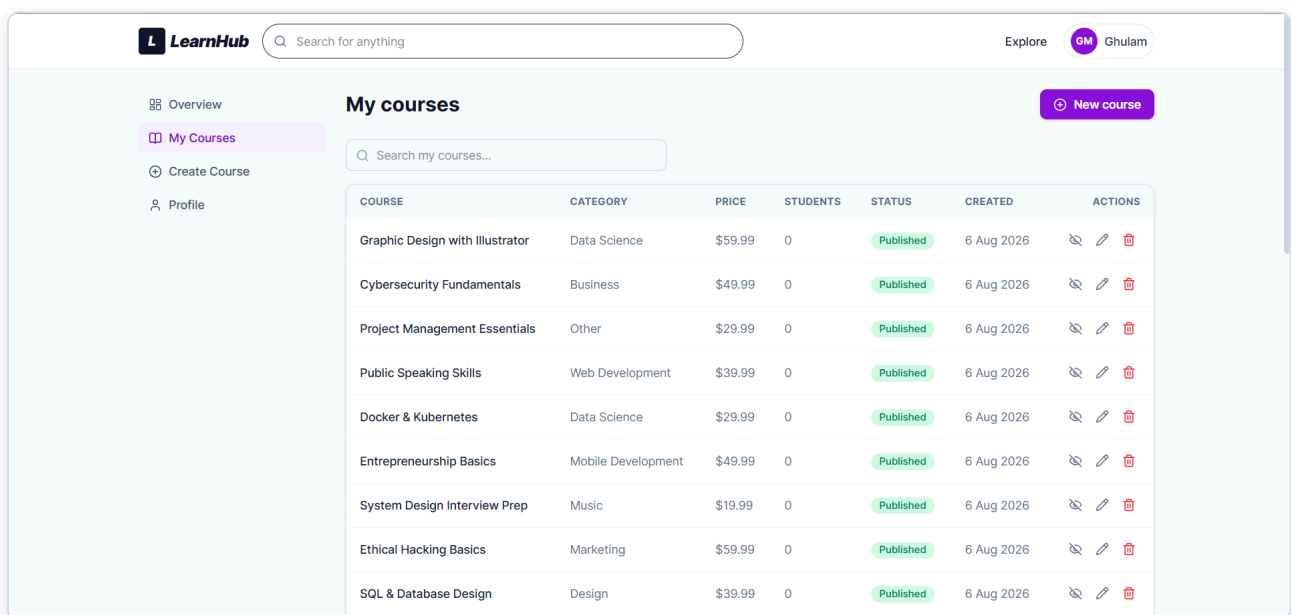


Figure 6.8 — Instructor's "My Courses" page, showing the searchable, paginated list of courses owned by the signed-in instructor with publish and edit controls.

The screenshot shows the 'Create a new course' form in the LearnHub application. The interface includes a top navigation bar with the LearnHub logo, a search bar, and a user profile dropdown for 'Ghulam'. A left sidebar contains navigation links: Overview, My Courses, Create Course (highlighted), and Profile. The main content area is titled 'Create a new course' and contains a form with the following fields: 'Course title' (text input), 'Description' (text area), 'Category' (dropdown menu with 'Web Development' selected), 'Level' (dropdown menu with 'Beginner' selected), and 'Price (USD)' (text input with '0'). A purple 'Create course' button is located at the bottom of the form.

Figure 6.9 — Course editor, showing the form used to create or edit a course and manage its ordered list of lessons, including title, content, video link, and duration.

The screenshot shows the 'Profile settings' page in the LearnHub application. The interface is similar to the previous one, with the 'Profile' link in the sidebar highlighted. The main content area is titled 'Profile settings' and contains two sections. The first section, 'Profile', shows the user's profile card with the initials 'GM', the name 'Ghulam Mohiyudin', the email 'ghulammohiyudin11@gmail.com', and the role 'instructor'. Below this are text inputs for 'Full name' (containing 'Ghulam Mohiyudin') and 'Bio' (containing 'Instructor account seeded with course catalog.'). A purple 'Save profile' button is at the bottom of this section. The second section, 'Change password', contains text inputs for 'Current password' and 'New password'.

Figure 6.10 — Instructor profile page, showing the account details view available to the signed-in instructor.

7. Challenges Faced

- **Vercel monorepo build root** — deploying straight from the repository root produced a platform-level 404 on every route, since the client and server live in separate subfolders and Vercel had no Root Directory configured to locate the frontend; resolved by setting the client as the project's build root so its own `vercel.json` SPA rewrite rules are respected.
- **Client silently calling itself instead of the API** — in production, login requests returned 405 Method Not Allowed because `VITE_API_URL` was unset at build time, leaving the Axios client pointed at a relative `/api` path on the frontend's own domain; the SPA rewrite then served `index.html` (a static file, GET-only) for the POST request. Fixed by setting `VITE_API_URL` to the deployed Render backend URL before rebuilding.
- **Cross-origin authentication cookies** — with the frontend and backend on different domains, the login cookie was not attached to API requests by default; this was traced to the cookie's `SameSite` attribute, which browsers do not send on cross-site requests unless it is explicitly set to `None`; Secure in production.
- **Enforcing ownership and role checks on both layers** — hiding instructor-only controls in the React UI was not sufficient by itself, since any authenticated user could still call the course API directly; every mutating course and lesson route also needed an `assertOwnerOrAdmin` check so instructors can only edit their own courses.
- **Embedding third-party lesson videos safely** — lesson `videoUrl` values could be entered in several different YouTube URL formats (`watch?v=`, `youtu.be/`, existing embed links); a small regex-based parser normalises any of these into a consistent `/embed/<id>` URL before it is placed in an `iframe`.
- **Sourcing real, relevant course media** — seeded course thumbnails initially used randomly generated placeholder images; these were replaced with real, topic-relevant images (technology logos and photographs) resolved from Wikimedia Commons and matching, real YouTube tutorial links for each of the 30 seeded course topics.

8. Conclusion

This task was completed successfully, delivering a full-stack Online Course Platform built with React, Node.js, Express, and MongoDB. All required features were implemented, including authentication (register, login, logout), a complete course module with ordered lessons (create, edit, publish, delete), a searchable and filterable public catalog with pagination, a simulated enrollment checkout for free and paid courses, and role-specific dashboards for students, instructors, and admins with client- and server-side validation. Both bonus requirements were

also delivered: all data is stored permanently in MongoDB, and role-based access control — student, instructor, and admin — is enforced independently on the client and the server, down to per-course ownership checks. The application was deployed end-to-end, with the API hosted on Render and the frontend on Vercel, and the monorepo build configuration, cross-origin cookie, and CORS issues that surfaced during deployment were diagnosed and resolved. This project reinforced practical, production-relevant skills in secure cookie-based authentication, role- and ownership-based authorization, REST API design with search and pagination, third-party media embedding, and responsive full-stack application deployment.